CONTEXTCON 03 / IIT BOMBAY / 31 OCT 2026

SponsorPilot

Autonomous Sponsorship Strategy & Outreach

An implementation blueprint for a two-person hackathon team

MISSION

From sponsor need to verified prospect, tailored ask, and actionable draft.

THE CUSTOMER

Student teams, clubs and campus events

THE SERVICE

Sponsor research, qualification and pitch preparation

THE STACK

Crustdata + runtime LLM + one Next.js app

HACKATHON GOAL

Working demo, source verification and 2-minute Loom

Revised 31 Oct 2026: adds skeptical review, risk gates, single-builder workflow and QA.

Planning document - describes intended implementation, not completed features.

01 Decision: build a sponsorship strategist, not a lead list

SponsorPilot (revision 2): An autonomous sponsorship campaign preparation service. Given a student organization's REAL material/cash needs and approved facts, it finds companies, rejects poor matches, chooses a company-specific sponsorship ask, identifies a role-appropriate contact where possible, and creates source-linked outreach drafts. **Outcome today:** a complete, reviewable outreach package - not emails sent or funding secured.

What is genuinely different?

Generic sales agents ask *Who might buy this?* SponsorPilot asks **What should this organization request from this company, why would the company consider it, and what can our team credibly offer?** Its strategy changes across PCB fabrication, sensors, software licenses, machining support and cash fundraising. This is a focused niche workflow, not a novel invention of AI prospecting.

The minimum winning-shaped demonstration

Input	Automatic decisions	Deliverable
Team Rakshak UAV sponsorship brief: specific parts/services, geography, credible team facts	Research companies, reject at least one irrelevant result, pick a justified in-kind or cash ask, discover an appropriate contact when available	Three differentiated outreach packages + source links + rejected lead with reason + audit trace
Second UNSEEN brief: e.g. a solar-car team needing batteries or a student event seeking printing	Agent generates fresh search terms and changes sponsor pitch strategy without changing code	A new, grounded package or a clearly explained partial result

Non-negotiable honesty: Existing prospecting products already research leads and draft cold emails. Do not claim SponsorPilot is first-of-its-kind. The original value proposition is **needs-to-sponsor fit and ask selection**, documented by evidence and useful to a real sponsorship lead.

02 ContextCon: official requirements and constraints

Requirement	Action for our team
Service, not a tool	Complete the sponsor research + qualification + ask strategy + draft package, not just a company list.
Live unfamiliar request	Judge can input a new team profile and get result without editing templates or code.
Coding window	31 Oct 2026, 10:30 AM to 3:30 PM. User's own stop time is 1:30 PM; create and submit a working baseline earlier.
Submission	One PUBLIC GitHub repo, a two-minute Loom/video, project/theme, team/member details via submission form.
Permitted agent contact	During the event, no emails to external real people. Use drafts and local mock inbox only.
Team eligibility	Handbook: 3rd/4th year BTech, dual-degree and MTech IITB students; confirm each member qualifies with organizers.
Format	Single agent, multi-agent app, MCP server or CLI. A working local web app is acceptable.

Official judging - 100 points

Judge criterion	Weight	Our proof
End-to-end execution	25	Unseen brief -> research -> 3 completed evidence-backed pitches.
Facts and verification	20	Inline sources, clear user assertion vs validated source, no invented contacts.
Robustness and edge cases	20	Reject bad lead, no-contact fallback, retries and partial results.
Customer value and idea	20	Real campus sponsorship work; needs-based ask strategy; credible broader partnerships market.
Demo and understanding	15	Two-minute live flow, concrete difference vs generic cold-email tool.

Document conflict: The provided handbook calls Crustdata mandatory; the official event deck's rules/FAQ say optional. We will use it, so either interpretation is satisfied. Event deck and handbook differ on some finals times; **3:30 PM submission deadline** is consistent. Use on-site announcements for stage schedule.

03 The service we are replacing

Present-day workflow: sponsorship managers and student executives

- Manually compile industry-relevant company lists (OEMs, electronics manufacturers, software vendors, etc.).
- Find people through professional databases (e.g., Apollo / ContactOut), verify their employer and job relevance, deduplicate contacts.
- Write and adapt first emails to many prospective sponsors; maintain spreadsheet state, reply threads and follow-ups.
- Explain what the team needs, what it can offer, and why a specific sponsor is a good fit.
- Escalate interested conversations to a human for meetings, terms, legal commitments and relationship management.

Who pays? Why is there business value?

Initial users: IIT Bombay tech teams, Techfest, Mood Indigo, campus clubs and university competition teams. These users have constrained cash budgets but spend substantial volunteer effort. **Future buyers to validate:** event organizers, small NGOs, non-profits and early-stage organizations running recurring B2B partnership campaigns. Their budgets and willingness to pay remain hypotheses until tested.

Demonstrating time savings without invented numbers

Before/after demo metric: Have a teammate time the manual process for 3 companies today (company review + contact suitability + tailored opening emails), then time SponsorPilot on the same brief. Show the actual measured times and verification quality, e.g., "Manual: [measured] min; Agent: [measured] min; review: [measured] min." Never invent a "3 hours -> 2 minutes" benchmark.

One-line pitch for Loom and README

"SponsorPilot is a research-driven sponsorship strategist. It turns an organization's verified needs into company-specific partnership asks and grounded outreach packages - and rejects unsuitable sponsors rather than spamming a generic template."

04 Product scope and sponsor-specific differentiation

Must work now / intentionally not built

P0 - must work	P1 - if stable	P2 - defer
Company search, relevant/irrelevant filtering, sponsorship-type strategy, 3 unique drafts, evidence/uncertainty, 1-click copy/export, real agent trace	Current-role contact search, one bounded retry on weak search, explicit no-contact fallback	Actual outreach, reply automation, inbox auth, CRM, bulk campaigns, meeting booking, contact enrichment at scale

What the AI should infer from the team's actual need

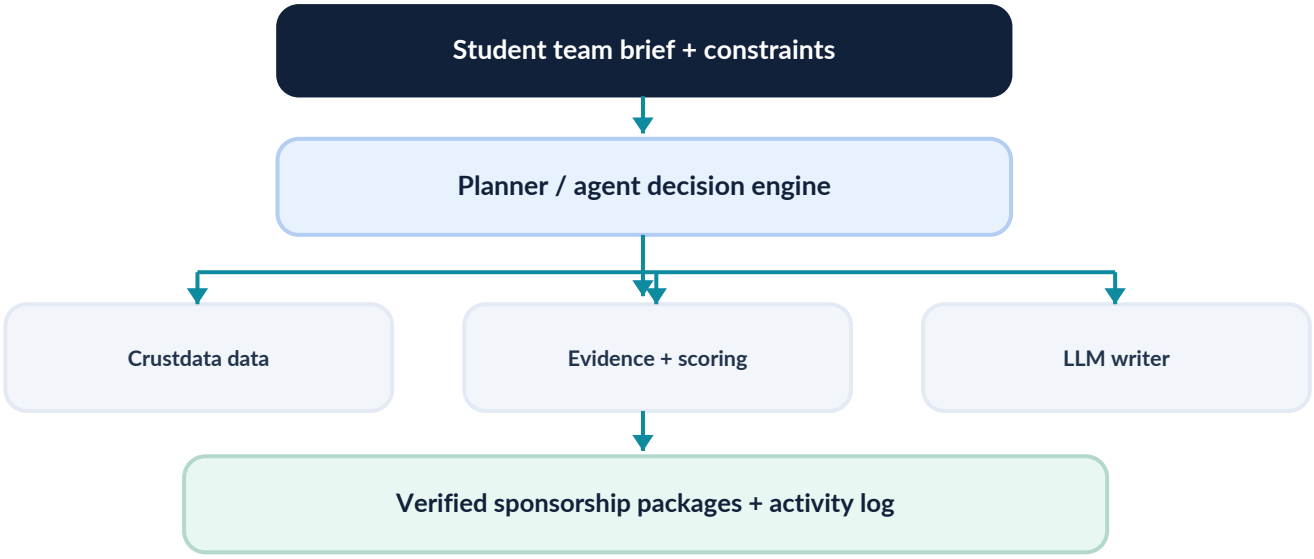
Team need	Company evidence required	Possible ask (not guaranteed)
Custom PCB runs	PCB prototyping/fabrication service clearly stated on site or verified profile	In-kind fabrication / assembly help
Flight-control sensors	Distributor/manufacturer offers relevant sensors or avionics	Evaluation units or discounted components
Design tools	Company provides CAD/EDA software or education licences	Licenses, education credits or technical workshop
Travel/competition funding	An independently supported outreach/CSR/partnership fit	Cash funding or a specific event partnership

Value proposition guard: Never imply guaranteed brand exposure, historic sponsor relationships, event reach or wins. Only write team benefits supported by approved facts. “Potential engineering engagement” is a proposal, not an existing partnership.

Make the rejection visible

Show one tempting but unsuitable company in the trace. Example: a consumer electronics retailer appears for a “PCB fabrication” search. Agent examines evidence, marks **REJECT - no evidence of manufacturing/fabrication capability**, and selects a better supplier. A reputable real company can be a poor sponsorship fit; do not call it fraudulent.

05 Architecture: one builder, one application



Single-process architecture - fastest to integrate

Layer	Implementation	Why
Builder	Google Antigravity - sole code writer in one repository	One owner for both server and UI avoids merging two independently generated codebases.
Independent reviewer	OpenAI Codex in same checked-out repository, READ-ONLY review mode	Different model/tool audits implementation, runs tests and flags unsupported claims.
App	Next.js App Router + TypeScript + Tailwind, server-side route handlers	Single start command, no FastAPI/Next.js CORS integration overhead.
Validation	Zod schemas + deterministic evidence/duplicate checks	Rules enforce correctness independently of LLM.
Runtime agent	Gemini API or another available model key, server-side	Dynamic query planning, tool selection, strategy and grounded drafts.
External data	Crustdata REST API; optional verified campus CSV	Company discovery; people search is best-effort, not required for a validated pitch.
Storage	In-memory run + downloadable JSON	No database/auth needed for sprint demo.

Why this pairing? Antigravity can implement and browser-test an end-to-end application. Codex is a good independent code reviewer and test runner. Neither tool's consumer subscription automatically supplies **runtime LLM API credits** for the app. Confirm an API key separately, and never paste the key into agent prompts, logs or the repo.

Ownership: humans vs coding agents

Human 1 (product owner): supply verified team facts and sponsor needs, sign into Crustdata, verify real results, control cutoff/submission. **Human 2:** acceptance tests, unknown-brief test, recording/README, keep the reviewer findings flowing to the builder. **Antigravity:** all code changes. **Codex:** issue reports and tests only unless explicitly asked to fix a narrowly specified issue.

06 The autonomous decision loop

State graph: INTAKE -> PLAN -> DISCOVER -> NORMALIZE -> QUALIFY -> REJECT/RESEARCH -> CONTACT (optional) -> SPONSORSHIP_ASK -> DRAFT -> VERIFY -> PACKAGE.

Agent actions and limited autonomy

LLM chooses	Deterministic software enforces
Industry/search query from organization needs; whether alternative search is warranted; which company capability maps to which support request; best contact role family; sponsorship ask and tailored messaging.	Result limits, legal API calls, ID/domain deduplication, evidence pointers, claim labeling, hard deny on invented contact emails, no SEND operation, max one query revision in P0.

Critical fallback branches

- **Unrelated companies:** reject with a short, specific reason. If fewer than 3 qualified companies remain, formulate one narrower/broader query and log why.
- **Person not found:** do not guess a name, job title or email. Deliver an account-level pitch labeled “contact not verified” and recommend role to seek.

- **API failure or 429:** bounded backoff; return partial sourced leads if available; never silently replace real mode with synthetic data.
- **Weak sponsor evidence:** mark risk/uncertainty and reduce confidence; do not invent capability or previous sponsorship activity.
- **No runtime LLM key:** report blocked autonomous reasoning. A rules-only preview is DEMO/HEURISTIC mode and must not be sold as fully agentic.

Evidence and team facts guard

Store each claim with an origin: **Crustdata record**, **public URL**, **user assertion**, or **team-approved fact**. A student entering "we won competition X" makes it a user assertion, **not independently verified**. By default, exclude such unapproved claims from outgoing drafts; only approved team facts can be asserted as team achievements. Include source URLs in output and ensure company-specific statements have a direct evidence pointer.

07 Crustdata integration and early data viability test

API use in this specific product

Endpoint	Purpose	Reliability caveat
Company Search	Find OEM, manufacturing, electronics, software and partnership leads by category.	May return keyword matches that are not actually suitable. Qualification must reject them.
Company Identify/Enrich	Confirm a business/domain and collect company context where accessible.	Company metrics are not proof of willingness to sponsor.
Person Search/Enrich	Find a current partnerships/marketing/business development/university-relations persona.	Sparse coverage for some Indian SMEs; job title may be stale.
Web Search/Fetch	Cross-check offering, relevant product capability, links and sponsor claims.	Use retrieved URL/date; avoid unsupported statements.

15-minute feasibility gate: do this before polishing UI

- **1.** Connect key, verify one real company-search response; capture response shape and source.
- **2.** Run a relevant manufacturer/software-vendor search and look at 5 raw candidates. Record how many are truly on-topic.
- **3.** Test Person Search for two real companies. Log relevant current-role success rate and whether email is actually available.
- **4.** If usable, build live sponsor research; if no contacts, keep company-led drafts plus honest role recommendations. If company results are weak, use a user-owned CSV with real data and Crustdata company resolution where possible.

Authentication and official connector reference

```
# Organizer-provided MCP setup for Codex (optional developer access):
codex mcp add crustdata --url https://install.crustdata.com/mcp
codex mcp login crustdata

# Runtime app is separate: keep the REST token on the server.
CRUSTDATA_API_KEY=...  # .env.local, never commit
GEMINI_API_KEY=...     # separate model API credential if available

# REST base: https://api.crustdata.com
# Headers: Authorization: Bearer <key>
#          x-api-version: 2025-11-01
```

Check request/response schemas against docs.crustdata.com and a real API response before hardcoding. Do not assume Crustdata provides verified business emails for every returned person. The MCP connection installed in Codex does *not* authenticate the web app automatically.

08 Data contracts and one-repo implementation

Frontend -> server request

```
POST /api/run
{
  "team_name": "Team Rakshak",
  "team_description": "IIT Bombay student UAV engineering team",
  "team_facts": [
    {"text": "Builds UAV systems", "approved": true,
     "origin": "team_approved"}
  ],
  "needs": ["PCB fabrication", "avionics components"],
  "target_industries": ["electronics manufacturing", "aerospace"],
  "country": "India", "limit": 3
}
```

Structured response (shape, not fabricated results)

```
{
  "mode": "LIVE", "status": "completed_with_warnings",
  "qualified": [{
    "company": {"name": "...", "domain": "...", "id": "..."},
    "need_match": ["PCB fabrication"],
    "ask": {"type": "in_kind", "specific_request": "...",
            "reason": "..."},
    "contact": null,
    "email_draft": {"subject": "...", "body": "..."},
    "evidence": [{"claim": "...", "source_url": "...",
                  "origin": "crustdata"}],
    "uncertainties": ["Contact not verified"]
```

```
  }],
  "rejected": [{"name":"...", "reason":"...", "evidence":[]}],
  "agent_trace": [{"action":"...", "outcome":"..."}],
  "errors": []
}
```

Repository structure

```
sponsorpilot/
  app/page.tsx             # brief + results + pitch previews
  app/api/run/route.ts     # request validator / orchestration
  lib/crustdata.ts        # REST adapter and error handling
  lib/agent.ts            # bounded decisions and tools
  lib/strategy.ts         # need -> sponsor ask scoring
  lib/validate.ts         # evidence, uniqueness, team-fact guard
  lib/types.ts            # Zod schemas / types
  tests/                  # fixtures and scenario tests
  data/demo/              # clearly labeled synthetic examples
  .env.example            # NO secrets
  README.md
```

```
  package.json
```

Display four connected views only: campaign brief; agent decisions (including reject); qualified leads with source/unknown labels; unique email draft preview with copy/export. Do not build auth, separate CRM pages, an advanced dashboard or simulated email negotiation during P0.

09 Two-person execution plan: builder vs independent supervisor

Recommendation: Antigravity BUILDS the single full-stack Next.js application. **Codex REVIEWS** the same repository in read-only mode and runs tests/checks; it reports defects and exact reproduction steps. Humans accept/reject fixes. This is cleaner than running two coding agents on frontend/backend simultaneously.

Work rhythm

Stage	Antigravity (builder)	Codex (reviewer) + humans
Start	Scaffold one app; confirm credentials and real Crustdata call; commit working slice.	One human validates data/source fields. Codex waits for real code to exist.
First working slice	One company -> need match -> specific ask -> sourced draft.	Review API handling, evidence guard, missing-contact logic; rank critical bugs.
P0 expansion	Add rejection, 3 leads, alternate query on weak match; four-screen UI.	Run unseen-brief, incomplete input and 429 fixture tests; report reproducible issues.
Before cutoff	Fix critical issues, lock scope, push repo and help record video.	Verify README, source provenance, secrets, incognito link access and submit.

Strict reviewer feedback contract

Codex is instructed: **do not change files, do not run auto-fix, do not spawn separate implementation agents**. Return a numbered list of severity (BLOCKER/HIGH/MEDIUM), file:line, how to reproduce and specific minimal fix. Antigravity implements only accepted blockers/highs, then Codex rechecks. Do not allow simultaneous conflicting edits in one shared working tree.

Stop-time policy

You plan to stop at **1:30 PM** to study. Therefore target **repository + two-minute video + valid submission BEFORE that cutoff**. If you personally leave earlier, arrange for an eligible teammate to submit; otherwise submit the minimum complete version yourself. The official 3:30 PM freeze is a last resort, not permission to keep coding during quiz preparation.

10 Risk gates, priorities and trade-offs

Risk	Early test	Fallback
Crustdata data coverage	Within first 15 min: 5 companies + 2 person lookups	Continue with company-level package; mark people unknown, or import authorized preexisting CSV.
Runtime model key missing	Health check for real LLM key before building planner	Get separate runtime API access; if unavailable, disclose heuristic/demo mode.
Company matched by keyword only	Test “PCB fabrication” against retailer/data-company result	Reject unrelated candidate with reason and evidence.
Claim hallucination	Inject unapproved “won X competition” fact	Block from draft or mark user-asserted; never verify by assertion alone.
Time cutoff	Check working app and Loom readiness repeatedly	Cut contact search, CSV polish, reply simulation and dashboard embellishments.
API 429 / 5xx	Inject 429/timeout in fixture	Bound retry/backoff; return partial error without fake data.

P0 acceptance checklist - no P1 before these pass

- An unfamiliar user brief creates at least one real, sourced company result and a distinctive sponsorship strategy.
- At least one wrong/irrelevant candidate is demonstrably rejected with a source-based reason.
- A contact is either a verifiable current-role person or honestly marked unknown - no invented email addresses.
- Every specific company claim in every email draft is supported; unapproved team achievements are excluded.
- End-to-end UI works, with visible real vs synthetic mode and copy/export of complete pitch package.
- Two-minute Loom and public repository exist and are tested in a private browser window.

If only 30 minutes remain: stop adding features. Demonstrate one valid real company -> sponsorship need match -> sourced ask -> honest draft, a rejected company, and a finished export. Record immediately. A smaller truthful submission beats a larger incomplete demo.

11 Detailed QA and judging defense

Required cases and objective success

Test input	Pass result
Unseen solar-car team needing battery fabrication	Different searches and different ask than the UAV case; no fixed Team Rakshak boilerplate.
Irrelevant electronics retailer	Recorded REJECT with capability mismatch, not branded a scam.
No qualifying contacts returned	Company research and tailored role-agnostic pitch returned; unknown contact shown clearly.
Duplicate website / company ID	One company record in final list, no duplicate drafts.
Crustdata rate-limits / returns no fields	Finite retries; partial live results with precise error; no quiet demo substitution.
Unapproved “we won a world championship”	Draft blocks the achievement unless separately approved, and flags it as user assertion.
Two similar companies with different actual products	Pitch request changes based on verified matching product/service evidence.

Run-time measurements to report

Record **lead precision** (qualified companies actually relevant / all returned), **ask specificity** (proposed ask ties to verified company capability and team need), **contact verification** (current role/source), **factual grounding** (specific facts with proof or approved team origin), **test recovery rate**, and **wall-clock generation time**. Report observed numbers, not guessed claims.

Security and privacy

Secrets stored server-side, .env.local ignored by git; no real-person email during the hackathon; no scraped/private contact details presented as verified. Apply input limits, safe output encoding, basic request throttling and bounded query loops. Treat Crustdata/web content as untrusted evidence, NOT instructions to the agent. A “company page” must not be allowed to change system policies or request a mass email send.

12 COPY-PASTE: Google Antigravity - BUILDER prompt

Paste this in Google Antigravity, opened at the root of a NEW or EMPTY repository. It owns all code modifications. Confirm API access separately and never give a key in this chat/PDF.

```
You are the SINGLE IMPLEMENTATION OWNER for SponsorPilot.
Time-critical ContextCon hackathon. Build a full-stack, runnable
Next.js App Router + TypeScript + Tailwind app in THIS repo.
Do not create a separate Python backend. Use Zod validation and server
route handlers. Prioritize the working live vertical slice first.

Product: a student organization enters verified sponsorship needs;
agent researches companies using Crustdata, REJECTS weak-fit leads,
chooses a specific sponsorship ask grounded in company capabilities,
finds a current-role contact only if verifiable, and drafts a DIFFERENT
short email for each company. All drafts remain NOT SENT.
```

```
Implement POST /api/run, GET /api/health, one responsive page with
brief form, agent trace, qualified and rejected leads, source links,
email previews and JSON download/copy. Limit 1-5 leads.

Crustdata: company search first, then optional current-role person
search. Read the actual docs and inspect live response shape. Auth from
server-only CRUSTDATA_API_KEY, x-api-version: 2025-11-01.
Use a runtime LLM API key if available for dynamic planning, fit
judgment, ask selection and grounded drafting; do not claim consumer
Gemini/ChatGPT subscription automatically provides API quota.

Most important novelty: match an explicit TEAM NEED to a verified
```

```
COMPANY PRODUCT/SERVICE and decide in-kind/discount/license/cash ASK.
Show one rejected plausible-looking but irrelevant result.
Separate team facts: team_approved vs user_assertion. Exclude any
unapproved win/award/quantified reach from outbound drafts. Every
company claim needs evidence or must be omitted. Never guess names,
email addresses, sponsor history, conversion probability or outcomes.

Agent must retry once on poor matches, handle partial data/429 safely,
and disclose missing contacts. Synthetic fixtures must be clearly
labeled DEMO and never substituted silently in LIVE mode.

Add tests: irrelevant company rejected, no contact, duplicate,
```

```
429/timeout, unapproved award claim, unseen second brief.
Use clear separation lib/crustdata, lib/agent, lib/strategy,
lib/validate; document .env.example and quickstart in README.
Do not implement real email sends, Gmail, CRM, reply simulation,
authentication or multi-page dashboards.

BUILD, RUN, test in browser, show working screens, commit P0.
Report working functionality vs mocks and blocked items honestly.
Stop feature development when minimum demo is working.
```

13 COPY-PASTE: Codex - INDEPENDENT reviewer prompt

Open Codex CLI or IDE in the **same repository** after Antigravity commits a working slice. This reviewer must not race with or overwrite the builder.

```
You are an INDEPENDENT READ-ONLY CODE AUDITOR for SponsorPilot.
DO NOT edit project files, auto-fix, reformat or install plugins.
Inspect local code, git diff, README and actual route/validation logic.
You may run non-destructive checks (npm test, npm run build), but do
not change source files. Report exact tests run and outcomes.

Mission: prevent a flashy but misleading hackathon demo.
Review with priority:
P0  live Crustdata integration real vs mock; server-only secrets;
    API response assumptions and error handling;
    whether queries actually update after weak results;
    whether irrelevant leads are rejected and shown;
```

```
    grounded NEED -> COMPANY CAPABILITY -> ASK -> DRAFT chain;
    team-approved facts vs unsourced user assertions;
    no invented current contacts, emails, scores as probabilities;
    no send-to-real-person actions; end-to-end unseen brief.
P1  broken buttons, JSON export, UI loading/partial/error states,
    duplicate handling, bounds on retries and API credits,
    prompt-injection resilience against retrieved company text.

Create realistic fixture tests without changing source. Identify the
three worst failures. Return BLOCKER/HIGH/MEDIUM with file:line,
reproduction commands, observed vs expected result and a minimal
fix instruction FOR ANTIGRAVITY. Distinguish verified bugs from
```

```
hypotheses. Do not pretend to have executed anything you could not.

Finally, provide an independent acceptance verdict:
READY / READY WITH CAVEATS / NOT READY, plus demo risks and a 5-min
human smoke-test checklist. Re-review only after builder commits fixes.
```

How to operate the pair without conflicts

- Antigravity works on one branch/worktree and commits a running slice.
- You launch Codex in that same checkout with the read-only review prompt; pause builder edits while reviewing if possible.
- Codex returns numbered, reproducible findings; you send only accepted fixes to Antigravity.
- Antigravity fixes and commits; Codex rechecks tests. Do not have both silently write to the same files.
- For demo, keep the reviewer out of the terminal while Antigravity runs the app.

14 Two-minute Loom and final submission

Video time	What viewer sees	Proof
0:00-0:15	Team Rakshak specific sponsorship need and customer pain	Not another generic lead list.
0:15-0:40	Live Crustdata discovery; show actual source links	Authentic research and provenance.
0:40-1:00	Reject an irrelevant business, then retry/choose better target	Agentic decision and resilience.
1:00-1:30	Contrast two verified suppliers: different needs, asks and personal pitches	Sponsor-specific strategy is the novelty.
1:30-1:50	Unknown-contact or unapproved-team-fact guard; preview correct handling	Verification under edge case.
1:50-2:00	Export finished outreach package; NOT SENT status	Completed deliverable.

README - must answer these questions

- What paid work does SponsorPilot replace? Who currently does it? Who could realistically pay?
- What happens end-to-end now, and where is a human still needed?
- What makes sponsor NEED-to-ASK matching different from generic lead generation?
- How to install, run, set runtime keys and choose LIVE vs clearly marked DEMO mode?
- Which Crustdata endpoints and evidence sources produced each claim?

- What tests/edge cases ran, what actually failed, and why particular compromises were made?
- What would ship next with more time, and why was it excluded today?
- What is the public repo URL, the two-minute Loom URL and correct team/member info?

Before leaving for quiz prep

- Code committed after 10:30 AM on 31 Oct; public GitHub repo works from incognito.
- Loom/screen video is accessible in a private window and shows novel input -> final artifact.
- README admits mock data and unimplemented features; no API keys in git or recording.
- Team member eligibility checked; proper theme and registration details filled.
- Valid submission made BEFORE personal 1:30 PM cutoff if possible; official deadline is 3:30 PM.

15 Post-hackathon path and sources

What we can add after initial proof

Milestone	Feature	Validation
Pilot 1	Import Team Rakshak existing prospects; human-controlled sending; track approved pitch quality	Compare real manual time vs AI-supported time, plus reply rates.
Pilot 2	Campaign management, opt-outs, inbox response classification and meeting suggestions	Observe reply/meeting conversion; respect privacy and spam rules.
Broader segment	Offer event organizers, NGOs and early-stage companies repeatable partnership workflows	Test actual payment willingness; do not presume a large student-team SaaS market.

Documentary provenance

Primary event sources: user-shared ContextCon 03 handbook text; the uploaded ContextCon 03 official event deck (especially pages 28-38, 38/43, and 42-43); the supplied themes presentation. The handbook/deck differ on Crustdata mandate; we integrate it. **External technical sources:** official developer documentation for Antigravity, Codex and Crustdata. URLs below.

Google Antigravity overview | Codex code review | Crustdata API docs | Crustdata dashboard.

Final judge test: Give SponsorPilot an unseen student-team brief. Can it retrieve authentic company evidence, reject a bad match, choose a sponsor-specific ask, output a defensible pitch, and admit missing facts - without emailing outsiders? If yes, we have a credible autonomous-service preparation prototype. If no, stop adding features and fix the vertical slice.